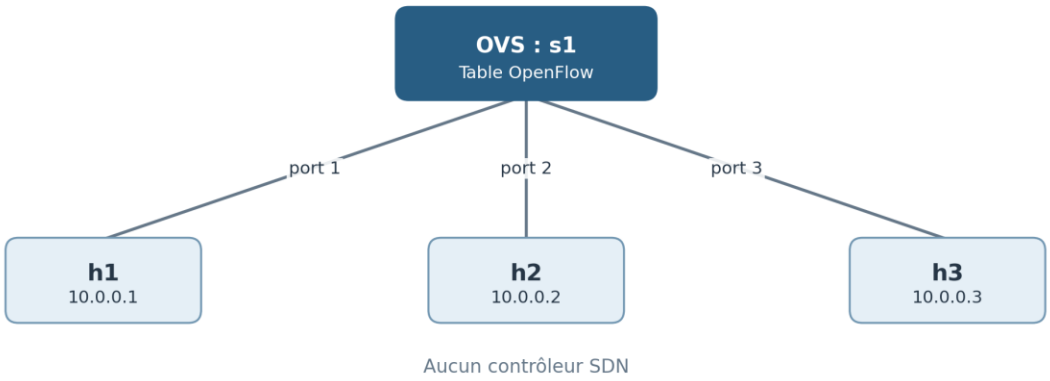
TP : Installation manuelle de flux OpenFlow dans Open vSwitch (OVS)

Dans ce TP il y a **6 ateliers progressifs d'installation manuelle de flux OpenFlow dans Open vSwitch (OVS)**, adaptés à un cours de niveau ingénieur SDN, avec Mininet, Ubuntu et OpenFlow 1.3.

L'objectif pédagogique est de montrer comment un switch OVS prend ses décisions de transfert **sans contrôleur SDN**, uniquement à partir des règles que nous installons manuellement avec `ovs-ofctl`.

1. Environnement de travail

Nous allons utiliser la même topologie pour tous les exemples.



Équipement	Adresse IP	Port OVS attendu
h1	10.0.0.1	s1-eth1 / 1
h2	10.0.0.2	s1-eth2 / 2
h3	10.0.0.3	s1-eth3 / 3

Initialisation

Dans un premier terminal Ubuntu :

```
sudo mn -c
sudo mn --topo single,3 --mac \
  --switch ovsk,protocols=OpenFlow13 \
  --controller none
```

Nous obtenons l'invite `mininet>`.

Dans un deuxième terminal Ubuntu :

```
sudo ovs-vsctl set-fail-mode s1 secure
sudo ovs-ofctl -O OpenFlow13 del-flows s1
sudo ovs-ofctl -O OpenFlow13 show s1
```

Pourquoi `secure` ? Sans ce paramétrage, OVS peut fonctionner en mode `standalone` et réaliser une commutation Ethernet classique. Le mode `secure` empêche ce comportement automatique : seuls les flux que nous installons sont utilisés pour le transfert.

La commande `show s1` permet de vérifier les numéros de ports OpenFlow avant les manipulations.

2. Exemple 1 — Installer deux flux pour permettre la communication entre h1 et h2

Objectif : autoriser h1 à communiquer avec h2, mais laisser h3 isolé.

Nous créons deux règles.

Dans le terminal Ubuntu :

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=100,in_port=1,actions=output:2"

sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=100,in_port=2,actions=output:1"
```

Explication des règles

Champ	Signification
priority=100	Priorité de la règle
in_port=1	Le paquet arrive sur le port 1
actions=output:2	Le paquet est envoyé sur le port 2
in_port=2	Le paquet arrive sur le port 2
actions=output:1	Le paquet est envoyé sur le port 1



Transfert bidirectionnel

Vérification

Dans Mininet :

```
mininet> h1 ping -c 3 h2
mininet> h1 ping -c 3 h3
```

Résultat attendu :

- h1 → h2 : communication réussie.
- h1 → h3 : communication impossible.

Interprétation : ces deux règles sont basées sur les ports d'entrée. Elles transfèrent aussi bien les paquets ARP que les paquets IPv4, puisqu'aucun protocole particulier n'est spécifié.

Remarque importante : il ne s'agit pas d'un apprentissage MAC. OVS ne cherche pas l'adresse MAC de destination ; il applique directement la correspondance entre ports.

3. Exemple 2 — Autoriser uniquement ICMP entre h1 et h2

Nous allons maintenant créer des règles plus précises.

Objectif : ne permettre que le protocole ICMP entre h1 et h2, tout en autorisant ARP pour la résolution des adresses MAC.

Étape 1 — Supprimer les anciennes règles

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

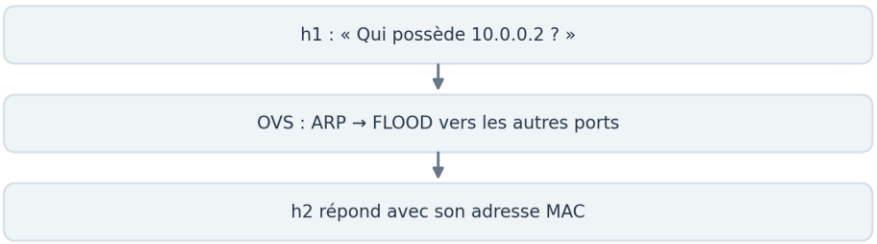
Étape 2 — Autoriser ARP

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=200,arp,actions=FLOOD"
```

Pourquoi ARP ?

Avant qu'h1 transmette son premier paquet ICMP à h2, il doit connaître l'adresse MAC de h2.

Le déroulement est:



Dans cette petite topologie sans boucle, l'action `FLOOD` facilite la démonstration. Dans un réseau de production, on peut préférer des règles ARP plus restrictives.

Étape 3 — Autoriser ICMP dans les deux sens

Flux h1 → h2 :

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=100,in_port=1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=output:2"
```

Flux h2 → h1 :

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=100,in_port=2,icmp,nw_src=10.0.0.2,nw_dst=10.0.0.1,actions=output:1"
```

Étape 4 — Bloquer explicitement tout le reste

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=0,actions=drop"
```

La table contient alors quatre règles :

Priorité	Match	Action
200	ARP	FLOOD
100	Entrée 1, ICMP, h1 → h2	output:2
100	Entrée 2, ICMP, h2 → h1	output:1
0	Tous les autres paquets	DROP

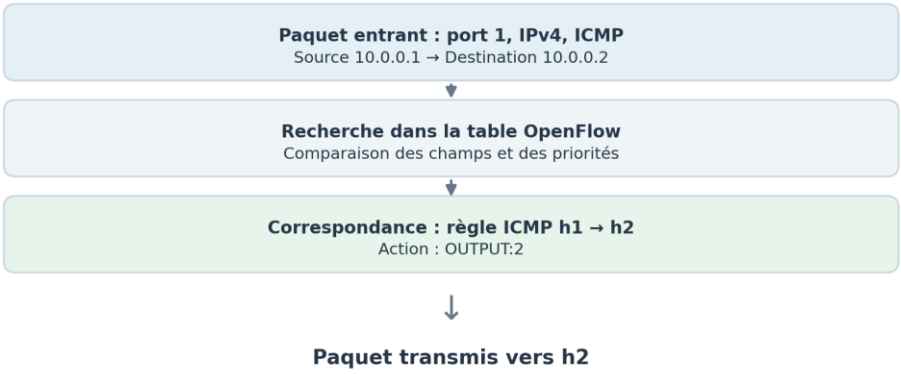
Étape 5 — Tester

Dans Mininet :

```
mininet> h1 ping -c 3 h2
mininet> h1 ping -c 3 h3
```

Le premier ping doit réussir, le second échouer.

Analyse technique d'un ping



Pour recevoir la réponse Echo Reply, nous avons besoin de la règle inverse.

> **Point important:** une règle OpenFlow classique est unidirectionnelle. Autoriser un paquet aller n'autorise pas automatiquement le paquet retour.

4. Exemple 3 — Modifier puis supprimer un flux OpenFlow

Reprenons la table installée à l'exemple 2.

Objectif : comprendre comment changer dynamiquement la politique du switch.

Visualiser les flux

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

La sortie comprendra des informations du type :

```
table=0, n_packets=6, priority=200,arp actions=FLOOD

table=0, n_packets=3, priority=100,
icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2
actions=output:2
```

Extrait illustratif : les compteurs réels et l'ordre des champs dépendent de l'exécution.

Les champs les plus utiles à analyser sont :

Champ	Signification
table=0	Table OpenFlow numéro 0
priority	Priorité de correspondance
duration	Temps écoulé depuis l'installation
n_packets	Nombre de paquets ayant correspondu
n_bytes	Nombre d'octets correspondants
actions	Traitement appliqué aux paquets

Modifier une règle pour bloquer ICMP

```
sudo ovs-ofctl -O OpenFlow13 --strict mod-flows s1 \
"priority=100,in_port=1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=drop"
```

Cette commande remplace l'action `output:2` par `drop`.

L'option `--strict` impose une correspondance exacte avec les champs de la règle, notamment sa priorité.

Test :

```
mininet> h1 ping -c 3 h2
```

Résultat attendu : échec.

Les paquets ARP peuvent toujours circuler, mais les requêtes ICMP h1 → h2 sont supprimées.

Rétablir la communication

```
sudo ovs-ofctl -O OpenFlow13 --strict mod-flows s1 \
"priority=100,in_port=1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=output:2"
```

Le ping doit fonctionner à nouveau.

Supprimer uniquement cette règle

```
sudo ovs-ofctl -O OpenFlow13 --strict del-flows s1 \
"priority=100,in_port=1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2"
```

La règle inverse h2 → h1 reste installée. Cependant, le ping ne fonctionne plus normalement puisque les requêtes h1 → h2 ne possèdent plus de règle autorisant leur transfert.

La syntaxe d'ajout, de modification et de suppression correspond à celle documentée par Open vSwitch.

5. Exemple 4 — Réaliser une ACL avec OpenFlow

C'est un exemple intéressant pour faire le lien avec les ACL Cisco.

Scénario : un administrateur souhaite autoriser les communications entre les machines, sauf le trafic IP de h1 vers h3.

Étape 1 — Réinitialiser

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

Étape 2 — Activer la commutation Ethernet classique comme règle de base

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=0,actions=NORMAL"
```

`NORMAL` demande à OVS d'effectuer sa commutation Ethernet traditionnelle (apprentissage MAC, transfert, etc.). C'est une action prise en charge par OVS, et non un simple `OUTPUT` vers un port fixe.

Vérifier :

```
mininet> pingall
```

Toutes les machines devraient communiquer.

Étape 3 — Ajouter une règle de blocage prioritaire

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=200,in_port=1,ip,nw_src=10.0.0.1,nw_dst=10.0.0.3,actions=drop"
```

Nous disposons désormais de deux règles.

Priorité	Match	Action
200	IP h1 → h3, port 1	DROP
0	Tous les autres paquets	NORMAL

Résultat attendu



Vérifier :

```
mininet> h1 ping -c 3 h3
mininet> h1 ping -c 3 h2
mininet> h2 ping -c 3 h3
```

Pourquoi la règle DROP gagne-t-elle ? Parce que sa priorité est 200, contre 0 pour la règle générale.

Remarque : cette ACL est stateless, c'est-à-dire sans suivi de session. Elle bloque les paquets IP h1 → h3, mais ne constitue pas à elle seule un pare-feu à états.

6. Exemple 5 — Autoriser uniquement un service HTTP sur le port TCP 8000

Voici un exemple plus proche d'une politique SDN de sécurité applicative.

Scénario : h2 héberge un serveur HTTP. Nous souhaitons autoriser h1 à accéder à ce serveur sur le port TCP 8000, mais aucun autre trafic IPv4.

Réinitialisation

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

Autoriser ARP

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=200,arp,actions=FLOOD"
```

Autoriser les requêtes TCP vers le serveur

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=100,in_port=1,tcp,nw_src=10.0.0.1,nw_dst=10.0.0.2,tp_dst=8000,actions=output:2"
```

Autoriser les réponses TCP du serveur

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=100,in_port=2,tcp,nw_src=10.0.0.2,nw_dst=10.0.0.1,tp_src=8000,actions=output:1"
```

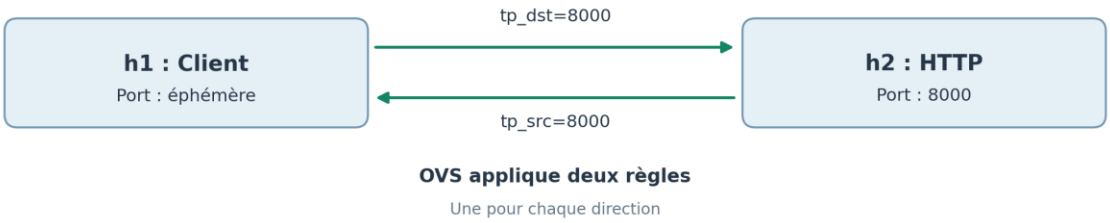
Bloquer les autres paquets

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=0,actions=drop"
```

Pourquoi deux règles TCP ?

Direction	Champ TCP examiné	Action
h1 → h2	Destination = 8000	output:2
h2 → h1	Source = 8000	output:1

Le client choisit généralement un port source éphémère. Le serveur répond depuis le port 8000 vers ce port éphémère.



Démarrer le serveur HTTP

Dans Mininet :

```
mininet> h2 python3 -m http.server 8000 --bind 10.0.0.2 &
```

Puis depuis h1 :

```
mininet> h1 curl http://10.0.0.2:8000
```

Résultat attendu : la page HTML du serveur est récupérée.

Tester maintenant :

```
mininet> h1 ping -c 3 h2
```

Le ping doit échouer, puisque seules les communications TCP correspondant aux règles précédentes sont autorisées.

Conclusion pédagogique : une politique OpenFlow peut être précise jusqu'aux champs de transport TCP/UDP, contrairement au premier exemple qui ne considérait que les ports physiques.

Attention : ces deux règles ne réalisent pas un véritable suivi de connexion TCP. Un filtrage à états nécessiterait des mécanismes supplémentaires, comme `conntrack`.

7. Exemple 6 — Gérer la durée de vie des flux

Dans les exemples précédents, les règles restent installées tant qu'elles ne sont pas supprimées ou que le switch ne les efface pas.

OpenFlow permet de définir des délais d'expiration.

Exemple :

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=150,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,idle_timeout=30,actions=output:2"
```

Le paramètre `idle_timeout=30` signifie que la règle expire après 30 secondes sans paquet correspondant. Le compteur d'inactivité est réinitialisé chaque fois qu'un paquet correspond à cette règle.

On peut utiliser :

```
hard_timeout=60
```

Dans ce cas, la règle expire après 60 secondes, même si elle continue à recevoir du trafic.

Paramètre	Comportement
idle_timeout=30	Suppression après 30 s d'inactivité
hard_timeout=60	Suppression après 60 s de durée de vie
idle_timeout=0	Pas d'expiration par inactivité
hard_timeout=0	Pas d'expiration liée à la durée totale

On peut également identifier les flux grâce à un `cookie` :

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"cookie=0x42,priority=150,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=output:2"
```

Puis supprimer uniquement les flux portant ce cookie :

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1 \
"cookie=0x42/-1"
```

Cette méthode est particulièrement utile pour gérer plusieurs politiques SDN indépendantes.

8. Synthèse des commandes OVS

Commande	Fonction
ovs-ofctl show s1	Afficher les ports OpenFlow
ovs-ofctl dump-flows s1	Consulter la table de flux

Commande	Fonction
ovs-ofctl add-flow	Ajouter une règle
ovs-ofctl mod-flows	Modifier une règle
ovs-ofctl del-flows	Supprimer des règles
ovs-ofctl dump-ports s1	Examiner les statistiques des ports
ovs-vsctl show	Visualiser la configuration OVS

Pour ces commandes OpenFlow, ajouter `-O OpenFlow13` comme dans les exemples précédents.

9. Observation avec Wireshark

Pour compléter le TP, il faut capturer sur `s1-eth1` ou `s1-eth2`, puis appliquer les filtres :

Expérience	Filtre Wireshark
Résolution des adresses	arp
Ping autorisé ou bloqué	icmp
Service HTTP	tcp.port == 8000
Établissement de connexion TCP	tcp.flags.syn == 1
Communication h1 → h2	ip.src == 10.0.0.1 && ip.dst == 10.0.0.2

La capture sur `s1-eth2` permet de vérifier si les paquets arrivent réellement au port de destination. En revanche, les commandes `ovs-ofctl` utilisant localement un socket Unix, leurs messages OpenFlow ne sont normalement pas visibles dans une capture TCP classique sur l'interface loopback.

Ce qu'il faut comprendre

Le fonctionnement fondamental d'OpenFlow peut se résumer à :

> **Paquet entrant → extraction des champs → comparaison avec les règles → sélection de la règle prioritaire → exécution des actions.**

Et surtout, un même switch OVS peut être programmé pour réaliser du switching, du filtrage ACL, de l'autorisation de services TCP/UDP ou de l'isolation réseau, sans modifier sa configuration physique.

Documentation Open vSwitch : https://www.openvswitch.org/support/dist-docs/ovs-ofctl.8.html

Documentation Mininet : https://mininet.org/walkthrough/